IaC Provisioning for Non-Profit System

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	Neha Tamboli	EN22CS301641
2	Riddhima Taose	EN22CS301642
3	Uday Patidar	EN23CS3L1023
4	Rohan Rawat	EN23CS3L1016
5	Naman Dwivedi	EN22CS301627

Group Name: 04D6

Project Number: DO-15

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Akshay Saxena

Academic Year: 2026

TABLE OF CONTENTS

1. **Problem Statement & Objectives**
   - 1.1 Problem Statement
   - 1.2 Project Objectives
   - 1.3 Scope of the Project
2. **System Design**
   - 2.1 Application Design
   - 2.2 Process Flow
   - 2.3 Information Flow
   - 2.4 Components Design
   - 2.5 Key Design Considerations
   - 2.6 API Catalogue
3. **Data Design**
   - 3.1 Data Flow Description
   - 3.2 Data Access Mechanism
   - 3.3 Data Retention Policies
   - 3.4 Data Migration
4. **Proposed Solution**
   - 4.1 Key Features
   - 4.2 Overall Architecture / Workflow
   - 4.3 Tools & Technologies Used
5. **Results & Output**
   - 5.1 Screenshots / Outputs
   - 5.2 Reports / Dashboards / Models
   - 5.3 Key Outcomes
6. **Testing & Validation**
7. **Limitations of the System**
8. **Conclusion**
9. **Future Scope & Enhancements**
10. **GitHub Repository**

1. Problem Statement & Objectives

1.1 Problem Statement

Non-profit organizations receive a large number of messages every day from donors, volunteers, and beneficiaries. These messages include requests for help, donation offers, complaints, and general inquiries. Handling such a large volume of messages manually is time-consuming, error-prone, and inefficient. Important messages may get delayed or ignored due to lack of proper prioritization.

Additionally, many organizations lack a structured system to categorize and process incoming messages effectively. This leads to poor communication management and reduced operational efficiency. Deploying and maintaining such systems manually also creates inconsistencies, delays, and scalability issues.

Therefore, there is a need for an intelligent system that can automatically classify, prioritize, and manage incoming messages while ensuring smooth deployment using modern DevOps practices.

1.2 Project Objectives

The main objectives of this project are:

- To develop an AI-based system that can automatically classify messages into categories such as donation, complaint, and request
- To detect the urgency level of messages (high, medium, low)
- To extract important information such as names, locations, and key entities from messages
- To reduce manual effort and improve response time
- To implement DevOps practices for automated deployment and monitoring
- To create a user-friendly interface for interacting with the system
- To ensure scalability and reliability using containerization and cloud platforms
- To provide insights through dashboards for better decision-making

1.3 Scope of the Project

This project focuses on building an intelligent message processing system for non-profit organizations. The system processes text-based inputs and classifies them using AI techniques. It also identifies urgency and extracts useful information from messages.

The project includes:

- Message classification and processing
- Deployment using Docker and AWS
- Automation using CI/CD pipelines

- Visualization through dashboards

However, the system is limited to text-based communication and does not currently support voice or image inputs. Future enhancements can expand its capabilities.

2. System Design

2.1. Application Design

The application follows an agent-based microservices architecture where specialized AI agents handle distinct processing tasks. The Classification Agent identifies message intent (donation, volunteer, complaint, general inquiry) and urgency level (high/medium/low). The Entity Extraction Agent identifies structured information including Indian names, ₹ amounts, locations, and dates. The Response Generation Agent creates contextual replies based on classification and extracted entities. These agents are orchestrated by LangChain framework and presented through a Streamlit web interface, all containerized within Docker for consistent deployment.

2.2 Process Flow

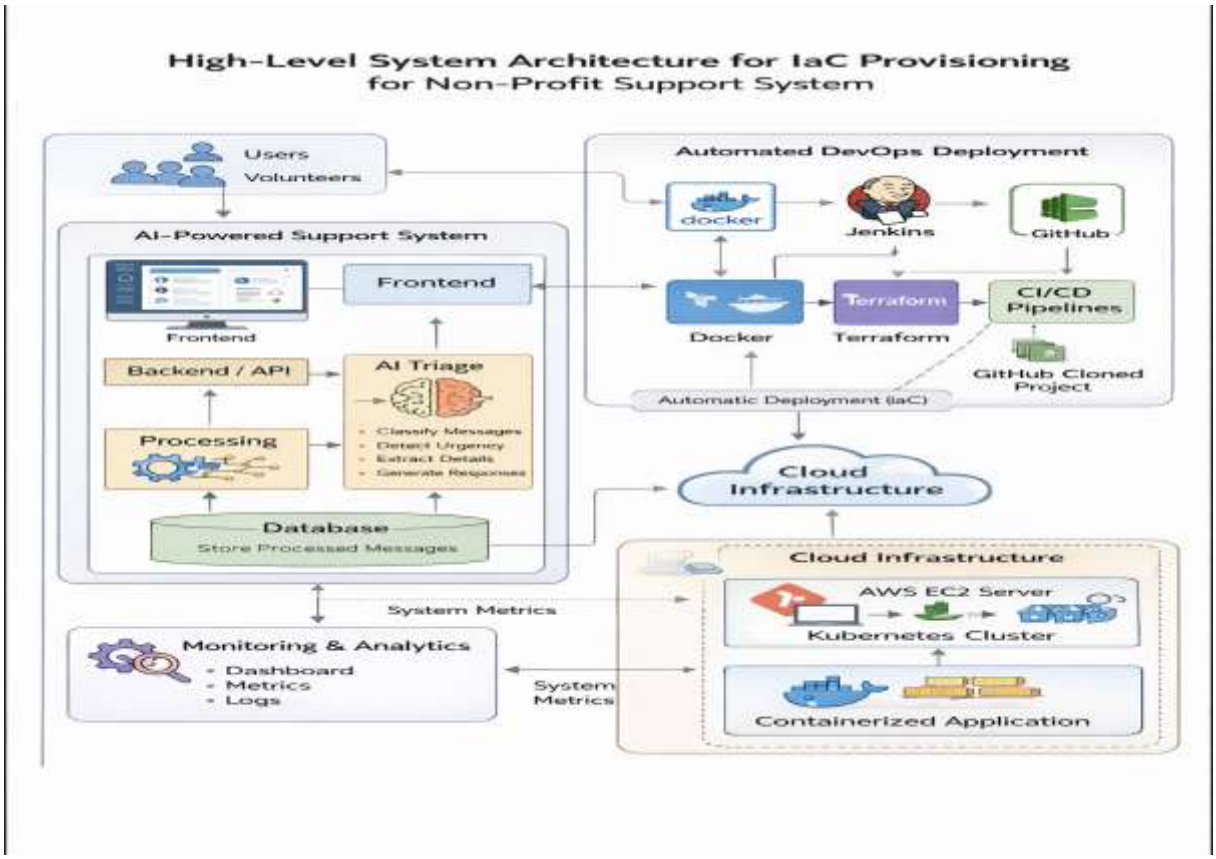
The process begins at the developer's machine, where the application code and Infrastructure as Code (IaC) scripts are developed and maintained. The code includes the AI-powered Non-Profit Support Triage system along with Docker configuration files for containerization.

Once development is complete, the code is pushed to a GitHub repository, which acts as the central version control system. This action triggers the CI/CD pipeline using tools such as GitHub Actions or Jenkins. The pipeline automatically performs tasks such as code validation, dependency installation, and Docker image creation.

After successful build and testing, the Docker image is deployed either on a local Docker environment or on cloud infrastructure such as AWS EC2. Terraform scripts can be used to provision the required infrastructure resources dynamically.

The application is then executed inside Docker containers, making it accessible through a web interface. The system processes incoming messages, performs classification and analysis, and generates responses. All processed data is stored and monitored through the system dashboard and analytics module.

This automated workflow ensures a consistent, scalable, and reliable deployment process while reducing manual intervention.



2.3 Information Flow

Information flows from the GitHub repository, where the code and IaC scripts are stored, to the CI/CD pipeline, which builds Docker images and provisions infrastructure using Terraform. The application is then deployed on Docker or cloud environments like AWS EC2. User messages are processed by the system, analyzed, and stored in the database, and the results are displayed on the dashboard. Terraform maintains a state file to track and manage the deployed resources.

2.4 Components Design

- **GitHub Repository:** Acts as the central version control system where application code, Docker configuration, and Infrastructure as Code (IaC) scripts are stored. It serves as the single source of truth and enables collaboration among team members.
- **CI/CD Pipeline (GitHub Actions/Jenkins):** Functions as the automation engine that triggers build and deployment processes whenever code is pushed to the repository. It handles tasks such as code validation, Docker image creation, and deployment automation.
-

- **Terraform:** Used as the core Infrastructure as Code tool to provision and manage cloud resources such as AWS EC2 instances. It ensures that infrastructure setup is automated, consistent, and repeatable.
- **Docker:** Used for containerizing the application, including the frontend and backend components. It ensures portability and consistent execution across different environments.
- **AWS EC2 / Cloud Environment:** Provides the hosting infrastructure where Docker containers are deployed and executed, making the application accessible via a public IP.
- **Application (Non-Profit Support Triage System):** The main system that processes user messages, performs classification, extracts information, and generates responses through an interactive dashboard.

2.5 Key Design Considerations

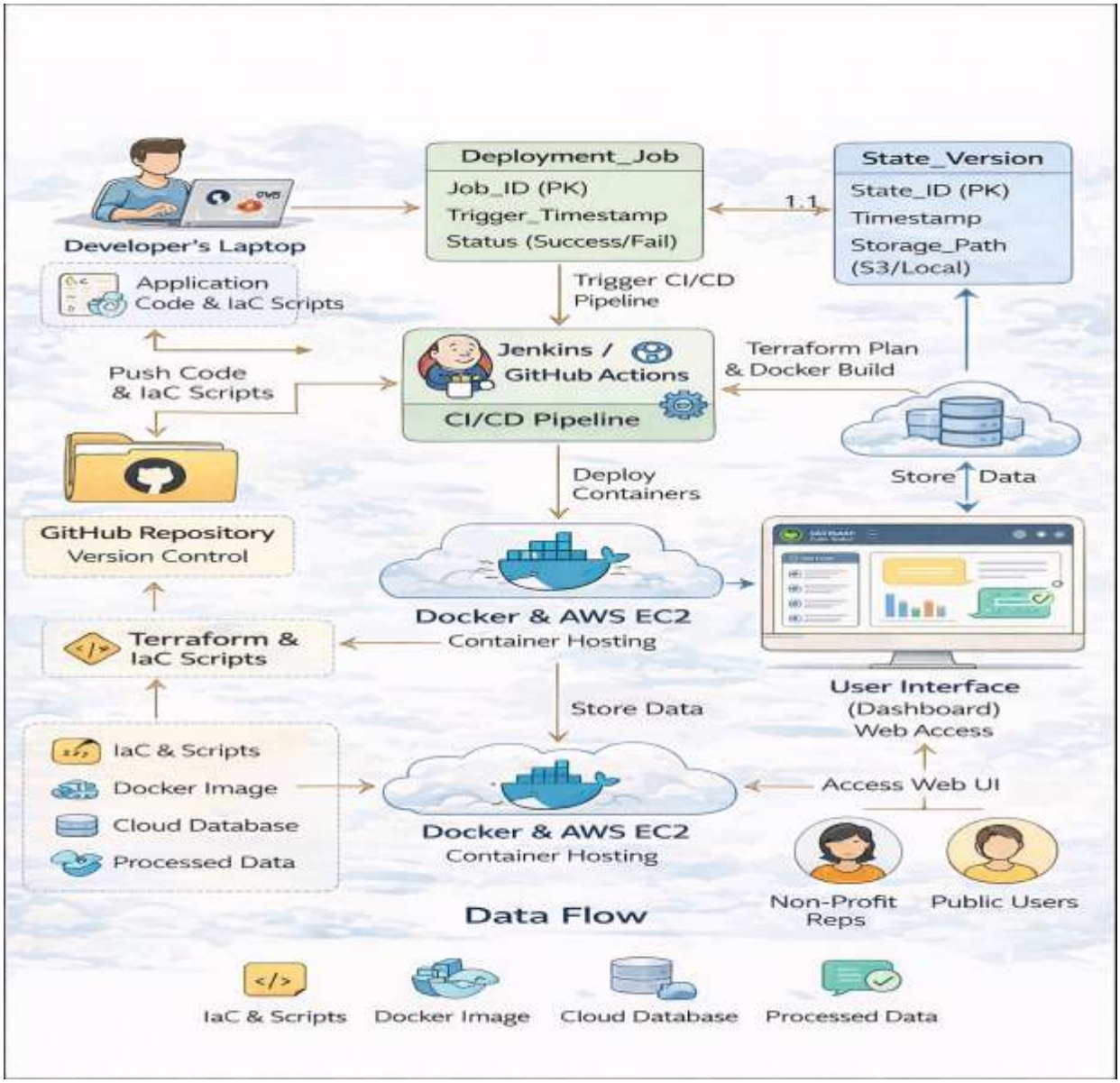
- **Modularity:** The system is designed using a modular approach where different components such as frontend, backend processing, AI logic, and infrastructure scripts are separated. This improves maintainability and allows easy updates or enhancements.
- **Scalability:** The application is containerized using Docker and can be deployed on cloud platforms like AWS, allowing the system to handle increasing user requests efficiently by scaling resources when needed.
- **Consistency:** By using Docker and Infrastructure as Code (Terraform), the system ensures that development, testing, and production environments remain identical, eliminating configuration mismatches.
- **Automation:** CI/CD pipelines automate the build and deployment process, reducing manual intervention and minimizing human errors.
- **Security:** Sensitive data such as API keys and credentials are managed using environment variables, ensuring secure handling of information.

2.6 API Catalogue

- **LLM API (Language Model API):** Used for processing user messages, including classification, entity extraction, and response generation in the Non-Profit Support Triage System.
- **Docker API:** Used to manage container lifecycle operations such as building images, running containers, and monitoring application services.
- **Cloud API (AWS EC2 API):** Used to provision and manage cloud infrastructure resources where the application is deployed.
- **CI/CD Pipeline API (GitHub Actions/Jenkins):** Used to automate build, test, and deployment processes whenever code changes are pushed to the repository.

3. Data Design

3.1 Data Flow Description



Data Flow

- The data flow of the system begins from the developer's machine, where application code and Infrastructure as Code (IaC) scripts are created. These scripts, along with the application code, are pushed to the GitHub repository, which acts as the central source of truth.
- Once the code is pushed, the CI/CD pipeline (GitHub Actions or Jenkins) is triggered automatically. The pipeline processes the code by building Docker images and executing Terraform scripts for provisioning infrastructure.
- The Docker containers are then deployed on cloud infrastructure such as AWS EC2. The application becomes accessible through a web-based user interface.
- Users interact with the system by submitting messages through the frontend. These messages are sent to the backend, where they are processed using the AI-based triage

- system. The system performs classification, urgency detection, and information extraction.
- The processed data is then stored in the database and displayed on the dashboard for monitoring and analysis. Additionally, Terraform maintains a state file to track infrastructure changes and ensure consistency between the code and deployed resources.

3.2 Data Access Mechanism

Access to code and infrastructure is managed through GitHub using role-based permissions. The CI/CD pipeline securely accesses the repository using tokens to build and deploy the application. Terraform uses secure credentials (AWS keys or environment variables) to provision resources. Application data is accessed through backend APIs with proper validation, and sensitive information is stored securely using environment configurations.

3.3 Data Retention Policies

- **Message Data Storage:** User messages and processed results are stored for a certain period to support analytics and monitoring of communication trends.
- **Deployment Logs:** CI/CD pipeline logs are maintained to track build and deployment activities for debugging and auditing purposes.
- **Infrastructure State:** Terraform state files are preserved to track infrastructure changes and support recovery in case of failures.
- **Security and Privacy:** Sensitive data is handled securely and can be removed or anonymized when no longer required.

3.4 Data Migration

Data and infrastructure migration in the system is managed using Infrastructure as Code (Terraform). By updating the provider configuration, the application and its resources can be moved from a local Docker environment to cloud platforms such as AWS EC2. Application data, including user messages and processed results, can also be migrated or backed up as needed to ensure continuity and system scalability.

4. Proposed Solution

4.1 Key Features

The proposed system includes the following key features:

- Automatic classification of messages into predefined categories
- Detection of urgency levels for prioritization
- Extraction of important entities from text
- User-friendly interface for message input and output

- Dashboard for monitoring system results
- Containerized application using Docker
- Automated deployment using CI/CD pipelines
- Cloud-based hosting

4.2 Overall Architecture / Workflow

The system follows a structured workflow starting from development to deployment and execution.

Initially, the developer writes the application code on a local machine. This code is then pushed to a Git repository. Once the code is pushed, it triggers an automated CI/CD pipeline.

The pipeline performs steps such as building the application, testing the code, and creating a Docker image. Infrastructure provisioning is handled using automation tools.

The Docker container is then deployed on a server where the application runs. Users interact with the system through a web interface. When a user submits a message, it is processed by the AI module which classifies the message, detects urgency, and extracts key information.

The processed results are displayed on a dashboard, allowing users to monitor and analyze the system output.

4.3 Tools & Technologies Used

The project uses the following technologies:

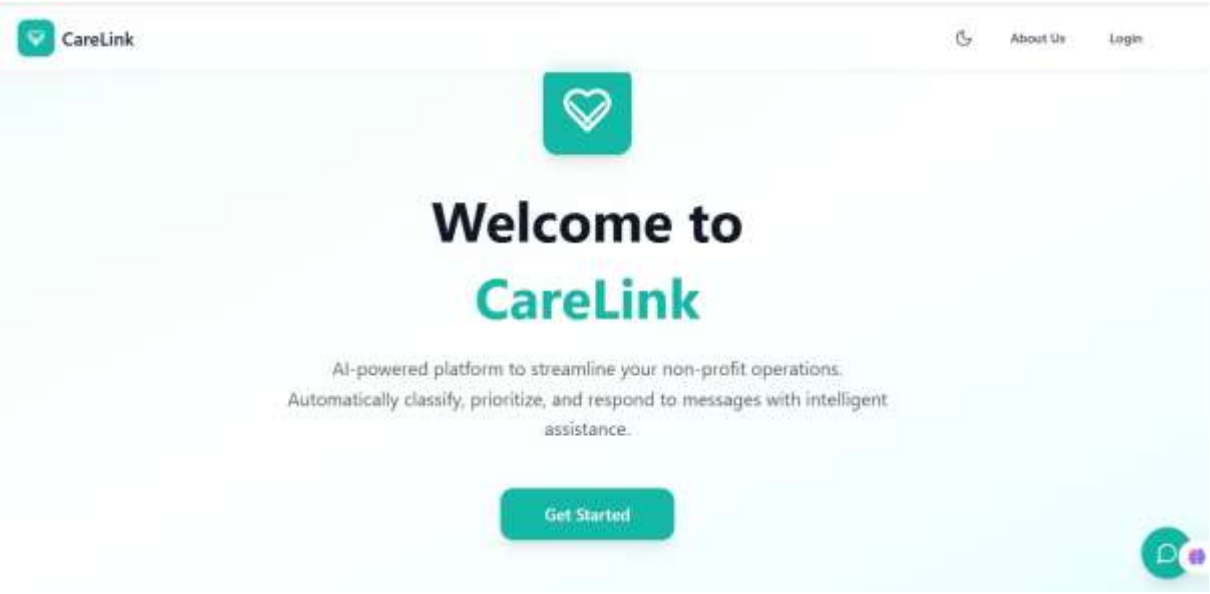
- Frontend: Web Interface
- Backend: Python
- AI/ML: Natural Language Processing techniques
- Containerization: Docker
- Infrastructure Automation: Terraform
- Cloud Platform: AWS
- Version Control: Git
- CI/CD: GitHub Actions or Jenkins

5. Results & Output

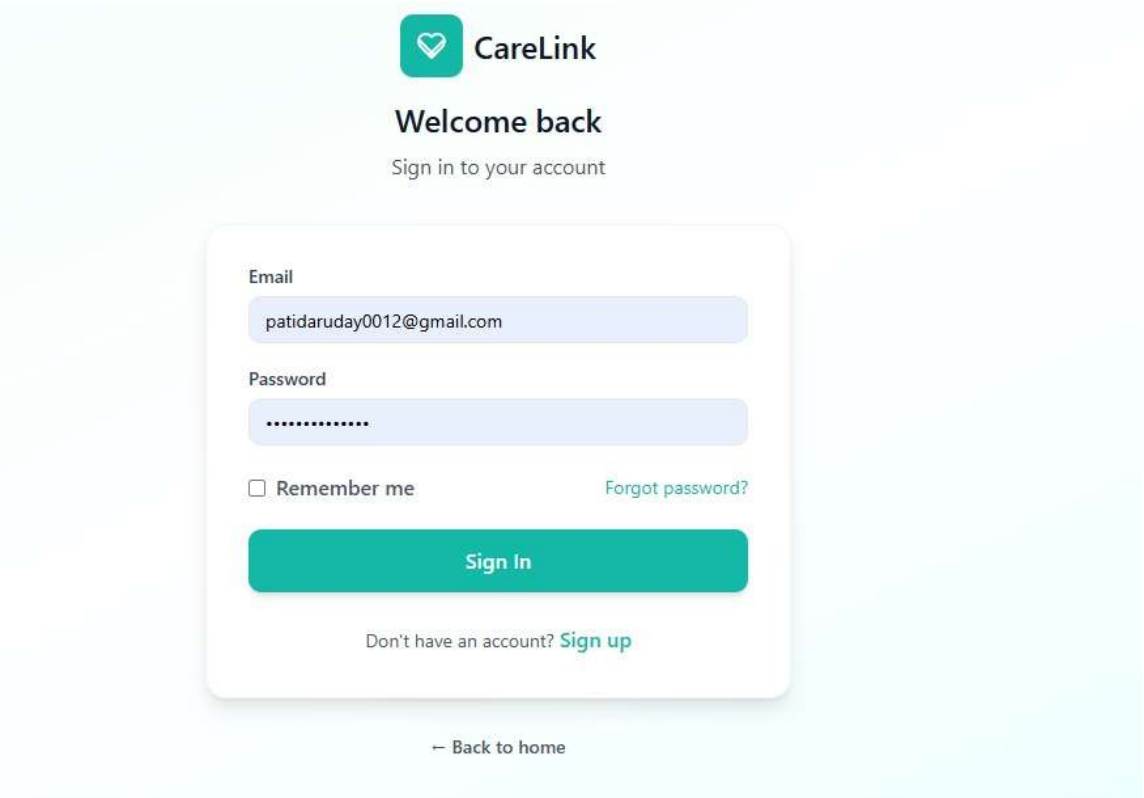
1. Screenshots of the project

❖ **User Interface – CareLink Home**

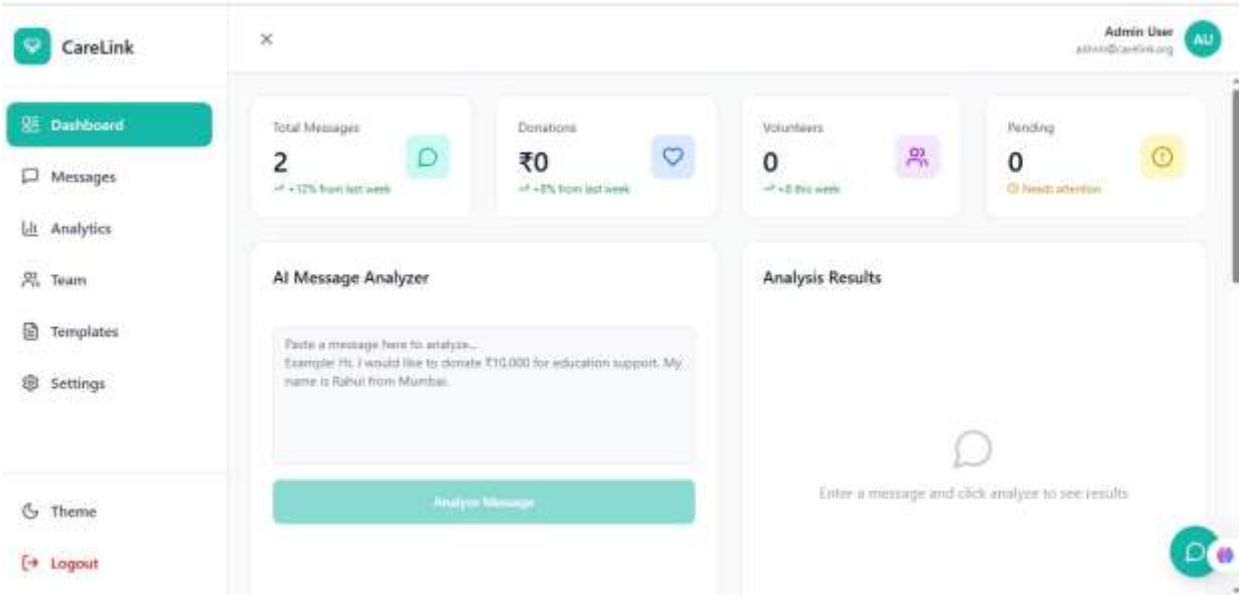
.



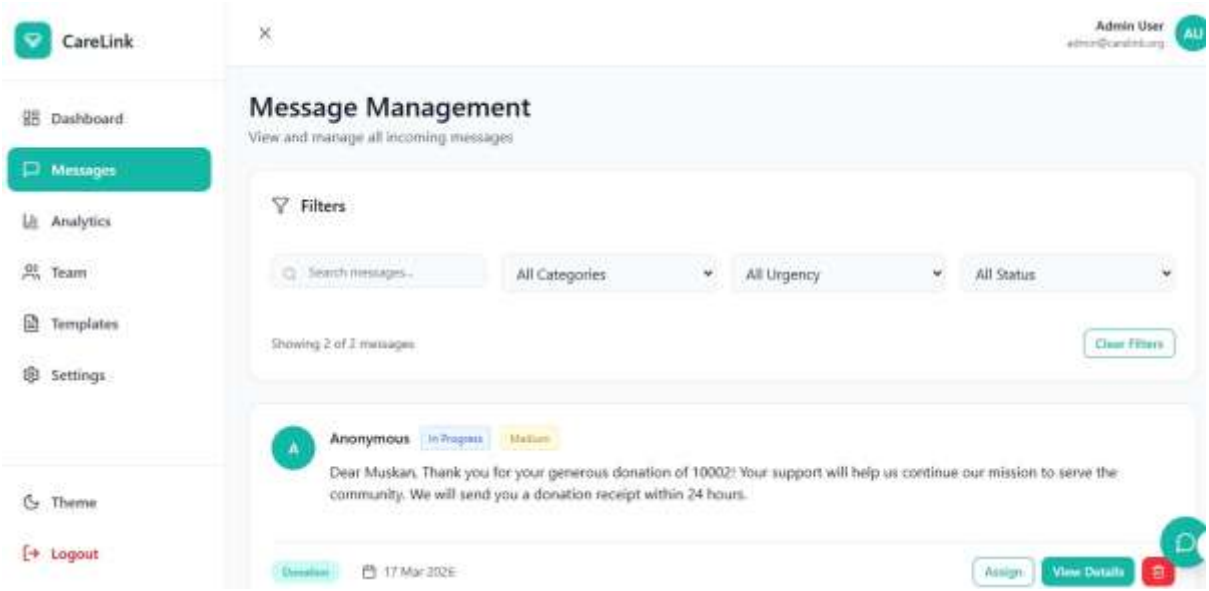
❖ **User Authentication – Login Page**



❖ **System Dashboard – CareLink Interface**

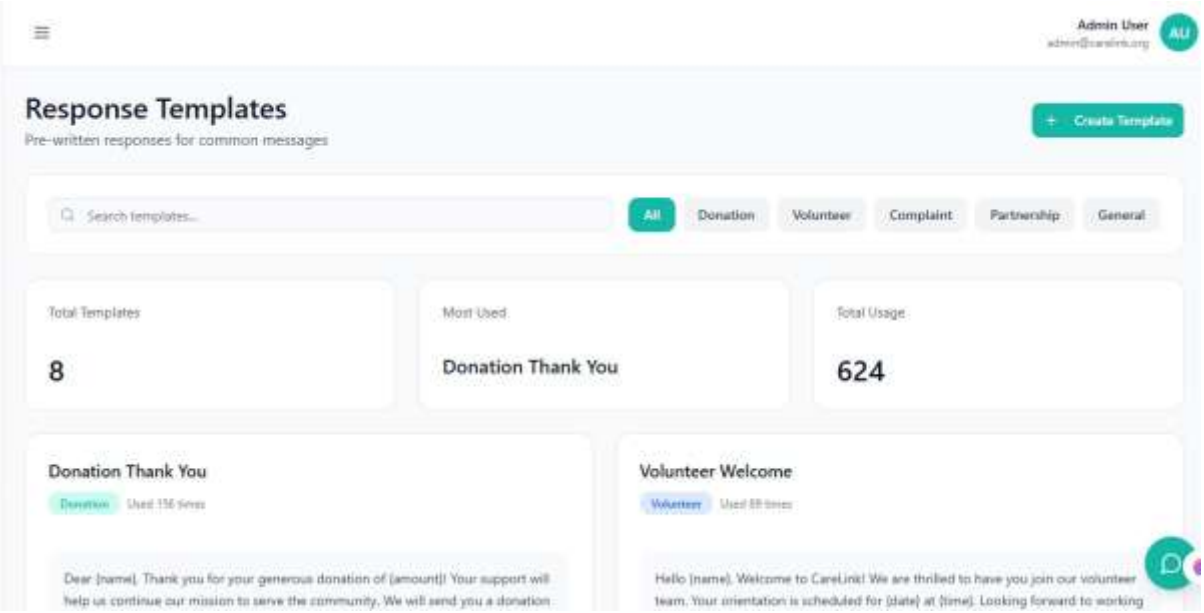


❖ **Message Management Module – CareLink Interface**



❖ **Analytics Dashboard – CareLink Interface**

❖ **Response Template Management – CareLink Interface**



5.2 Reports / Dashboards / Models

The system generates structured outputs in the form of categorized messages, urgency levels, and extracted entities. The dashboard presents this information visually, helping users understand patterns and trends.

5.3 Key Outcomes

The project achieves several important outcomes:

- Automation of message classification and prioritization
- Reduction in manual workload
- Faster response time for critical messages
- Improved accuracy using AI techniques
- Efficient deployment using automated processes
- Scalable system architecture

6. Testing & Validation

The system is tested to ensure accuracy and reliability. Different types of messages are used as test inputs to verify classification and urgency detection.

Performance testing ensures the system can handle multiple requests efficiently.

Advantages of the System

- Reduces manual effort
- Improves efficiency and accuracy
- Provides real-time insights
- Scalable and reliable
- Easy to deploy and maintain

7. Limitations of the System

- Limited to text-based input
- Accuracy depends on AI model quality
- Requires internet connectivity for deployment

8. Conclusion

This project demonstrates the integration of Artificial Intelligence and modern deployment practices to solve real-world problems. The system automates message classification and prioritization, improving efficiency and reducing manual workload.

The use of containerization and automation ensures smooth and reliable deployment. The project highlights the importance of intelligent systems in managing communication effectively.

9. Future Scope & Enhancements

The project can be further improved by adding support for voice and image inputs, improving AI model accuracy, integrating databases for better data management, developing a mobile application, and implementing real-time notifications.

10. GitHub - https://github.com/NehaTamboli/carelink-project